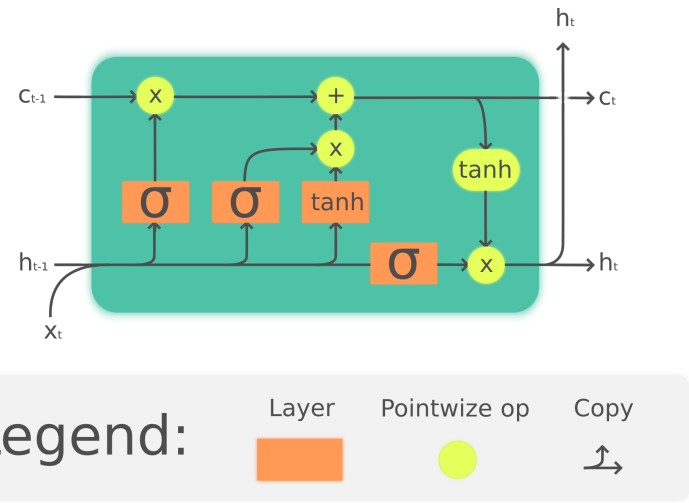
Lesson 10: LSTM Sequence Classifier

Companion diagram



LSTM cell diagram

Image source: https://commons.wikimedia.org/wiki/File:The_LSTM_cell.png

```
Goal
----
Understand sequence modeling with an LSTM in a way that feels concrete, not mysterious.

This lesson focuses on:
- token embeddings
- sequence order
- hidden state
- cell state
- final sequence summary

Why sequence models exist
-------------------------
Some data has an order:
- text
- audio
- time series
- sensor readings

Order matters.

For example:

    [I am not happy]

is different from:

    [I am happy not]

Sequence models try to respect that order.
```

Why LSTMs were important

Before Transformers became dominant, LSTMs were one of the main tools for sequence learning.

They are still useful for learning because they teach a strong mental model:

```
read tokens one step at a time
keep some memory
update that memory as new tokens arrive
```

Pipeline

```
token ids -> embedding -> LSTM -> final hidden state -> classifier
```

Token ids and embeddings

Input tokens have shape:

```
(batch, seq_len)
```

If:

```
batch = 64
seq_len = 10
```

then input shape is:

```
(64, 10)
```

After embedding with size 32:

```
(64, 10) -> (64, 10, 32)
```

Meaning:
- 64 sequences
- 10 positions per sequence
- 32 features per token after embedding

What the LSTM returns

The LSTM returns:

```
outputs, (hidden_state, cell_state)
```

This confuses many beginners, so separate them clearly.

`outputs`

Shape:

```
(batch, seq_len, hidden_size)
```

This gives one output vector for every time step.

`hidden_state`

Shape:

    (num_layers, batch, hidden_size)

This stores the final hidden state for each layer.

In a one-layer LSTM:

    hidden_state[-1]

has shape:

    (batch, hidden_size)

That is why the script uses it as the sequence summary.

`cell_state`

Shape:

    (num_layers, batch, hidden_size)

The cell state is part of the LSTM's internal memory mechanism.

You do not need every gate equation yet.
For now, keep this intuition:

```
hidden state = current exposed summary
cell state   = longer-term internal memory
```

Why sequence order matters here

The toy labels compare the average of the first half and second half of the sequence.

So:
- the same tokens in a different order can change the label
- the model must care about position over time

That is exactly the kind of problem LSTMs are built for.

Why we use the final hidden state

For sequence classification, we want one vector representing the whole sequence.

The final hidden state is one common choice because it summarizes what the model remembers after reading all time steps.

So:

    final_hidden = hidden_state[-1]

has shape:

    (batch, hidden_size)
Then the classifier maps:

```
    (batch, hidden_size) -> (batch, num_classes)

Common beginner mistakes
------------------------
1. Confusing `outputs` with `hidden_state`
2. Forgetting that embeddings add a feature dimension
3. Forgetting sequence order is the main point
4. Not understanding why one final vector is needed for classification
5. Expecting LSTM tensors to use the same shapes as CNNs

How this differs from Transformers
----------------------------------
LSTM:
- reads step by step
- carries hidden memory forward through time

Transformer:
- mainly uses attention across the whole sequence
- processes token relationships more directly

You should learn both intuitions.

Exercises
---------
1. Print shapes for `embedded`, `outputs`, `hidden_state`, and `cell_state`.
2. Replace `hidden_state[-1]` with `outputs[:, -1, :]` and compare.
3. Increase `hidden_size` from 32 to 64.
4. Change the label rule so the class depends on the first token only.

Self-check questions
--------------------
1. Why do token ids need an embedding layer before the LSTM?
2. What is the difference between `outputs` and `hidden_state`?
3. Why is the final hidden state useful for sequence classification?
4. What does the LSTM remember that a plain linear layer would not?
5. Why is order important in this task?
```

Read the matching Python file and run it while comparing the printed shapes, losses, and outputs against this handout.